

AI/ML Recruitment Task - First Year

Auto MPG dataset

Name: _____ Date: _____

This submission uses simple Python code and short explanations. I used the Auto MPG data to look at fuel efficiency and then tried to predict MPG with linear regression. The notebook code is written for Google Colab.

How to use this PDF: copy the code sections into Colab in order and run them from top to bottom. The graphs, printed statistics, and model scores will appear when the notebook is run. The cleaned CSV is saved at the end of Task 1.

Task 1: Exploratory Data Analysis and Preprocessing

1. Load the data

I load the Auto MPG data from a public copy of the dataset. The car name is kept for checking records, but it will not be used as a model feature later.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
df = pd.read_csv(url)

print(df.head())
print("Rows and columns:", df.shape)
print(df.info())
print(df.describe())
```

2. Check and clean the data

I check missing values and duplicate rows. In this dataset, some horsepower values can be missing. I fill those with the median horsepower because the median is less affected by very large or small values. I remove exact duplicate rows. I also make sure the numeric columns have numeric data types.

```
print("Missing values before cleaning:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

df = df.drop_duplicates().copy()
df["horsepower"] = pd.to_numeric(df["horsepower"], errors="coerce")
df["horsepower"] = df["horsepower"].fillna(df["horsepower"].median())

numeric_cols = ["mpg", "cylinders", "displacement", "horsepower",
                 "weight", "acceleration", "model_year", "origin"]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df = df.dropna(subset=["mpg", "weight"])
print("Shape after cleaning:", df.shape)
print("Missing values after cleaning:")
print(df.isnull().sum())

# Save the cleaned data for Task 2
df.to_csv("auto_mpg_cleaned.csv", index=False)
print("Saved auto_mpg_cleaned.csv")
```

3. Explore the columns and MPG

The columns describe fuel efficiency (mpg), engine size (displacement and cylinders), horsepower, vehicle weight, acceleration, model year, place of origin, and car name. MPG is the value I want to understand and predict.

```
print("MPG summary:")
print(df["mpg"].describe())

plt.figure(figsize=(6, 4))
```

```
sns.histplot(df["mpg"], bins=20, kde=True)
plt.title("Distribution of MPG")
plt.xlabel("Miles per gallon")
plt.show()

cols = ["displacement", "horsepower", "weight", "acceleration"]
df[cols].hist(figsize=(9, 6), bins=20)
plt.tight_layout()
plt.show()
```

4. Compare MPG with vehicle details

These scatter plots help me see whether MPG changes with engine size, horsepower, weight, and acceleration. The box plots compare MPG for different cylinder counts and model years.

```
for col in ["weight", "horsepower", "displacement", "acceleration"]:
    plt.figure(figsize=(5, 3.5))
    sns.scatterplot(data=df, x=col, y="mpg")
    plt.title("MPG compared with " + col)
    plt.show()

plt.figure(figsize=(6, 4))
sns.boxplot(data=df, x="cylinders", y="mpg")
plt.title("MPG by number of cylinders")
plt.show()

plt.figure(figsize=(8, 4))
sns.boxplot(data=df, x="model_year", y="mpg")
plt.title("MPG by model year")
plt.show()
```

5. Correlation and unusual values

A correlation close to 1 means two numeric values tend to increase together. A correlation close to -1 means one tends to decrease as the other increases. Correlation does not prove that one thing causes another. The box plot is a simple way to look for possible unusual MPG values.

```
corr = df[numeric_cols].corr()
print(corr["mpg"].sort_values(ascending=False))

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation between numeric columns")
plt.show()

plt.figure(figsize=(5, 3.5))
sns.boxplot(y=df["mpg"])
plt.title("Possible unusual MPG values")
plt.show()

# Print the rows with the lowest and highest MPG
print(df.sort_values("mpg").head(3)[["mpg", "weight", "horsepower", "car_name"]])
print(df.sort_values("mpg").tail(3)[["mpg", "weight", "horsepower", "car_name"]])
```

Task 1 continued

6. My own question

My question is: do newer model years tend to have higher MPG in this dataset? I compare the average MPG for earlier and later model years. These groups are only a simple comparison, so other differences between cars may also matter.

```
middle_year = df["model_year"].median()
df["year_group"] = np.where(df["model_year"] <= middle_year,
                             "Earlier years", "Later years")
print(df.groupby("year_group")["mpg"].agg(["mean", "median", "count"]))

sns.boxplot(data=df, x="year_group", y="mpg")
plt.title("MPG in earlier and later model years")
plt.show()
```

7. Findings to check after running the notebook

The output from the cells above should be used to complete these findings with the actual values from Colab. I have not filled in made-up measurements.

1. The cleaned dataset has _____ rows and _____ columns. I removed _____ duplicate rows and filled _____ missing horsepower values.
2. The average MPG is _____ and the MPG values range from _____ to _____.
3. The scatter plot shows that weight and MPG have a _____ relationship.
4. The plot suggests that horsepower and MPG have a _____ relationship.
5. Cars with _____ cylinders have a higher/lower median MPG than cars with _____ cylinders.
6. The later-year group has _____ average MPG than the earlier-year group.
7. One possible unusual record is _____. I checked its other values and _____.
8. My extra question was about model year. Based on the group averages, I conclude that _____.

The graphs and printed summaries above support these findings. The cleaned file `auto_mpg_cleaned.csv` is created by the cleaning code.

Task 2: Linear Regression

1. Prepare training and testing data

I predict MPG. First I use only vehicle weight, because it is an easy starting point. Then I use several numeric vehicle details and one-hot encoded origin. I leave car_name out of the features. The data is split into 80% training data and 20% testing data with random_state 42.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# If needed, load the CSV saved in Task 1
# df = pd.read_csv("auto_mpg_cleaned.csv")

# Single feature model: weight only
X_weight = df[["weight"]]
y = df["mpg"]
Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_weight, y, test_size=0.2, random_state=42)

weight_model = LinearRegression()
weight_model.fit(Xw_train, yw_train)
weight_pred = weight_model.predict(Xw_test)
print("Weight-only test R2:", r2_score(yw_test, weight_pred))

# Multiple features. Origin is treated as a category.
features = ["cylinders", "displacement", "horsepower", "weight",
            "acceleration", "model_year", "origin"]
X = df[features].copy()
X["origin"] = X["origin"].astype("category")
X = pd.get_dummies(X, columns=["origin"], drop_first=True, dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

2. Evaluate the model

MAE is the average size of the prediction error in MPG. MSE gives larger errors more weight. RMSE is the square root of MSE, so it is in MPG units. R-squared describes how much of the variation in MPG is explained by the model; closer to 1 is usually better.

```
def show_scores(actual, predicted, label):
    mae = mean_absolute_error(actual, predicted)
    mse = mean_squared_error(actual, predicted)
    rmse = np.sqrt(mse)
    r2 = r2_score(actual, predicted)
    print(label)
    print("MAE:", round(mae, 2))
    print("MSE:", round(mse, 2))
    print("RMSE:", round(rmse, 2))
    print("R2:", round(r2, 2))

show_scores(y_train, train_pred, "Training scores")
show_scores(y_test, test_pred, "Testing scores")
```

3. Plot actual and predicted MPG

```
plt.figure(figsize=(5, 5))
plt.scatter(y_test, test_pred)
plt.xlabel("Actual MPG")
plt.ylabel("Predicted MPG")
plt.title("Actual MPG and predicted MPG")

# A good prediction would be near this diagonal line
low = min(y_test.min(), test_pred.min())
high = max(y_test.max(), test_pred.max())
plt.plot([low, high], [low, high], color="red")
```

```
plt.show()
```

4. Model findings

Run the code first and copy the values into the notes below. If training scores are much better than testing scores, the model may be overfitting. If both are poor, it may be underfitting or missing useful patterns. Similar scores do not prove the model is perfect; the test set is only one sample.

1. The weight-only model got a test R-squared of _____. This means weight by itself _____.
2. The multiple-feature model had test MAE _____ MPG and test RMSE _____ MPG.
3. Its training R-squared was _____ and testing R-squared was _____.
4. Based on the training and testing scores, I think the model shows _____ (little/some) overfitting.
5. In the actual versus predicted plot, the predictions _____ (stay near / often differ from) the diagonal line.
6. One result I found interesting was _____.

Task 2 continued

5. Limitations and next step

1. Linear regression assumes mostly straight-line relationships, but real vehicle data may have more complicated patterns.
2. Some vehicle details may be related to each other, which can make it harder to understand each coefficient.
3. The dataset is not very large and may not represent all cars.
4. Some important information about the cars may not be included.

A possible next step is to compare this model with a decision tree model using the same train/test split and metrics. I would also check the data carefully and explain the results from the notebook outputs.

How I would submit

The recruitment brief asks for a Google Colab notebook, a shareable link with viewing access, and the cleaned dataset. After copying and running the code in Colab, I would check that every cell runs from top to bottom, save the notebook, download `auto_mpg_cleaned.csv`, and submit both as instructed by the recruitment team.

End of submission